

Rapport d'activité – Rendu 3

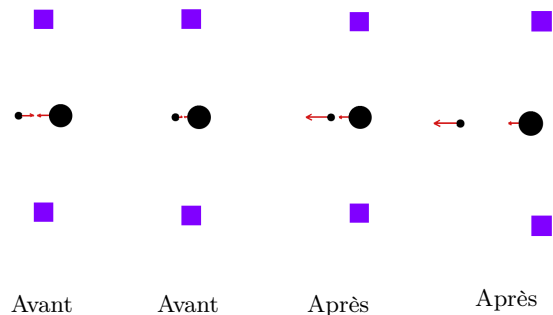
Partie 1 — Dynamiques du jeu

1.1 Changements structurels par rapport au Rendu 2 (optionnel)

- **Ball** : ajout des méthodes `translate()` et `setDelta()` pour rendre la classe mutable.
- **Paddle** : ajout de l'attribut `last_delta` et de la méthode `getDelta()` pour la collision balle-raquette.
- **Brick** : ajout de la méthode virtuelle pure `hit()` dans la hiérarchie de classes.
- **Game** : réécriture complète de `update()` avec les trois phases de la dynamique et toutes ses sous-fonctions.

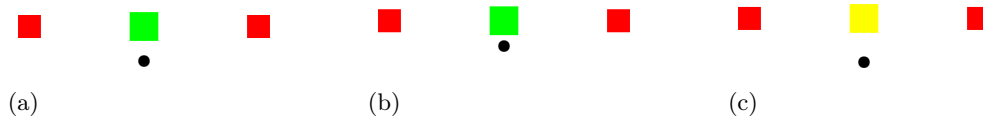
1.2 Collision de deux balles de rayons différents

Dans `update()`, `detect_collisions()` parcourt toutes les paires et signale un contact dès que la distance entre centres est $< r_a + r_b + \epsilon$. `bounce_ball()` projette alors le delta de chaque balle sur l'axe des centres (produit scalaire `dot`), calcule les facteurs d'échange $f_i = r_b / (r_a + r_b)$ et $f_j = r_a / (r_a + r_b)$, puis met à jour les deltas via `setDelta()` : la composante normale de A augmente, celle de B diminue. Selon l'intensité, B peut inverser sa direction ou simplement ralentir. Les composantes tangentielles restent inchangées. Les balles sont ensuite séparées (`translate()`) et leurs deltas bornés à `delta_norm_max`.

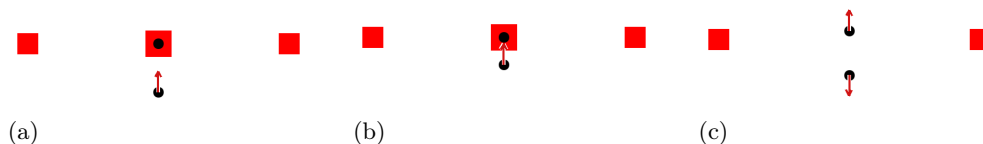


1.3 Destruction de chaque type de brique

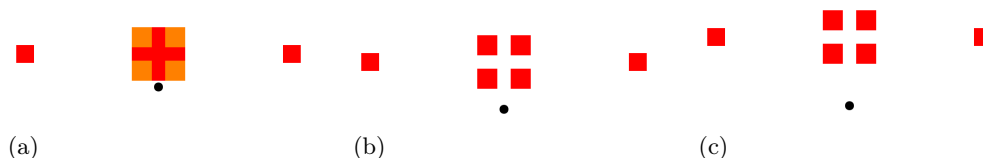
RainbowBrick : `bounce_brick()` détermine la face touchée en comparant les recouvrements horizontal et vertical, réfléchit le delta de la balle via `setDelta()`, puis appelle `hit()` : le compteur `hit_points` est décrémenté et l'index de couleur est incrémenté modulo la taille du cycle. Quand `hit_points` atteint zéro, `hit()` renvoie `true` et la brique est supprimée : (a) état initial, (b) après un impact, (c) détruite.



BallBrick : (a) brique intacte. (b) `bounce_brick()` sauvegarde `old_delta = ball.getDelta()` avant toute réflexion, réfléchit la balle frappante, puis appelle `hit()` : la brique est immédiatement détruite (`hit()` renvoie `true`) et un **Ball** centré sur la brique est poussé dans `new_balls` avec `old_delta`, capturant ainsi la direction avant rebond. (c) la balle frappante continue avec son delta réfléchi; la nouvelle balle part dans la direction `old_delta`, c'est-à-dire celle d'avant le rebond. Les deux sont intégrées au vecteur `balls` à l'itération suivante.



SplitBrick : (a) brique intacte. (b) `bounce_brick()` appelle `hit()` sur la brique mère : la taille fille $s' = (\text{size} - \text{split_brick_gap})/2$ est calculée. Si $s' \geq \text{brick_size_min}$, quatre nouvelles **SplitBrick** sontinstanciées aux quatre quadrants et ajoutées via `bricks.push_back()`. (c) les filles intègrent la liste active et peuvent à leur tour se diviser; si $s' < \text{brick_size_min}$, elles sont simplement détruites à l'impact sans spawner.



Partie 2 — Travail en groupe et bilan

2.1 Activité individuelle

- **Liam Van Camp** : *[Décrivez votre contribution — modules, fonctions, tests, etc.]*
- **Victor Eder** : *[Décrivez sa contribution — modules, fonctions, tests, etc.]*

2.2 Méthodologie

Organisation et outils : *[Décrivez comment vous vous êtes organisés — Git (branches, commits), VSCode Live Share, réunions, etc.]*

Ordre de développement et tests : *[Dans quel ordre avez-vous développé les modules? Ex : tools → ball/brick/paddle → game : :load → game : :update. Quelles méthodes de test pour chaque partie (fichiers .txt manuels, débogage visuel, etc.) ?]*

Travail simultané vs indépendant : *[Quelle proportion du code a été écrite côte à côte (pair programming) vs en parallèle de façon indépendante? Avec le recul, changeriez-vous cette proportion, et pourquoi ?]*

2.3 Bug le plus fréquent

[Décrivez le bug qui revenait le plus souvent — ex : balle traversant les briques (effet tunnel), boucle infinie dans update(), mauvaise direction nominale, etc. Quelle était la cause, comment l'avez-vous détecté et résolu ?]

2.4 Usage d'un outil d'IA

[Avez-vous utilisé un outil d'IA (Claude, ChatGPT, Copilot, etc.) ? Si oui, pour quelles tâches (compréhension des formules de collision, débogage, structuration du rapport, etc.) ? Gain ou perte de temps — quantifiez approximativement (ex : “gain d'environ 1h sur la compréhension de la formule d'impulsion balle-balle”).]

2.5 Conclusion

[Auto-évaluation courte sur deux axes :]

Code : *[Robustesse (gestion des cas limites, boucles infinies évitées via nb_bounce_max, etc.) et performance (complexité des boucles dans update(), etc.)]*

Environnement de travail : *[Points forts et points faibles de l'environnement fourni (GTKmm, structure imposée, fichiers de test, etc.) et améliorations que vous suggéreriez.]*